

Student Performance Predictor

AI/ML Project

Course: Fundamentals of AI & ML

Submitted as part of: VITyarthi - Build Your Own Project

NAME-Tanushri Ankush Halmare

Registration Number- 25MIB10058

1. Introduction

This report documents the design, implementation, and evaluation of the **Student Performance Predictor**, a machine learning project built for the Fundamentals of AI & ML course. The system predicts whether a student is likely to Pass or Fail based on four everyday indicators: study hours, attendance percentage, previous academic score, and average sleep hours. The project demonstrates the full ML workflow — data preparation, exploratory analysis, model training and selection, evaluation, and prediction — implemented as a modular, testable Python application.

2. Problem Statement

Educators often lack an early, data-driven way to identify students who are at risk of failing. Manually tracking study habits, attendance, and past performance for every student does not scale, and problems are frequently noticed too late for meaningful intervention. This project addresses that gap with a lightweight prediction tool that flags at-risk students early using a handful of simple, easily collected indicators.

3. Functional Requirements

- **Data Input & Processing:** generate/clean a structured student dataset, with a defined feature set (study hours, attendance %, previous score, sleep hours) and target (Pass/Fail).
- **Model Training & Prediction:** train and compare multiple classifiers, automatically select the best performer, and expose a clear prediction interface (input: 4 student attributes -> output: Pass/Fail + confidence).
- **Reporting & Visualization:** run exploratory data analysis, evaluate the trained model with standard classification metrics, and persist all results (charts, reports, model) to disk.
- A logical, linear workflow: load data -> clean -> split/scale -> EDA -> train -> evaluate -> predict.

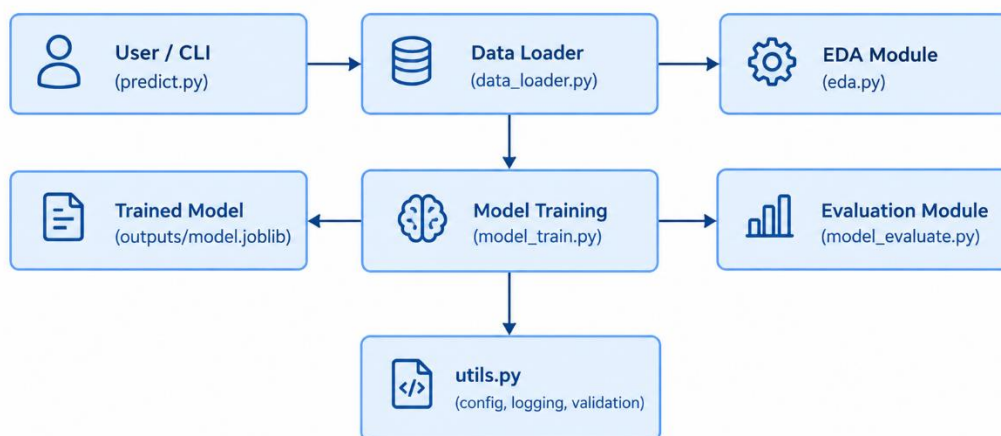
4. Non-Functional Requirements

- **Performance:** the entire pipeline (500-row dataset, 2 models) trains and evaluates in under a few seconds on a standard machine.
- **Reliability:** the dataset is generated with a fixed random seed, so results are reproducible across runs.
- **Usability:** a simple command-line interface lets a non-technical user enter four numbers and get an immediate, readable prediction.
- **Maintainability:** the codebase is split into single-responsibility modules (data, EDA, training, evaluation, prediction, utils), making it easy to extend or swap components.
- **Error handling:** all user-supplied prediction inputs are validated (type and range checks) before being passed to the model, with clear error messages on invalid input.
- **Logging/Monitoring:** every module logs its key actions and results via a shared, consistently formatted logger.

5. System Architecture

The system follows a layered, modular architecture. The Data Layer is responsible for generating and preparing data; the Model Training module trains and persists the classifier; the Evaluation module measures its performance; and the CLI / Predictor layer exposes predictions to the end user. A shared utilities module centralizes configuration, logging, and input validation used across all layers.

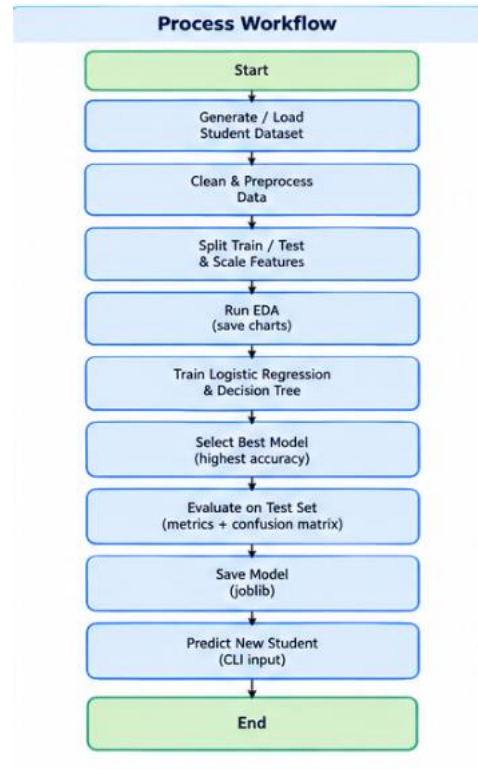
System Architecture



6. Design Diagrams

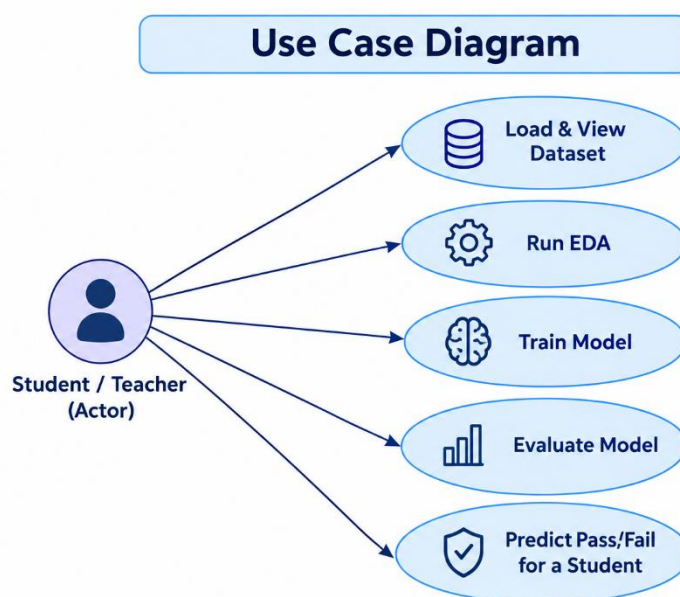
6.1 Process / Workflow Diagram

The diagram below shows the end-to-end execution flow of main.py, from data generation through to a sample prediction.



6.2 Use Case Diagram

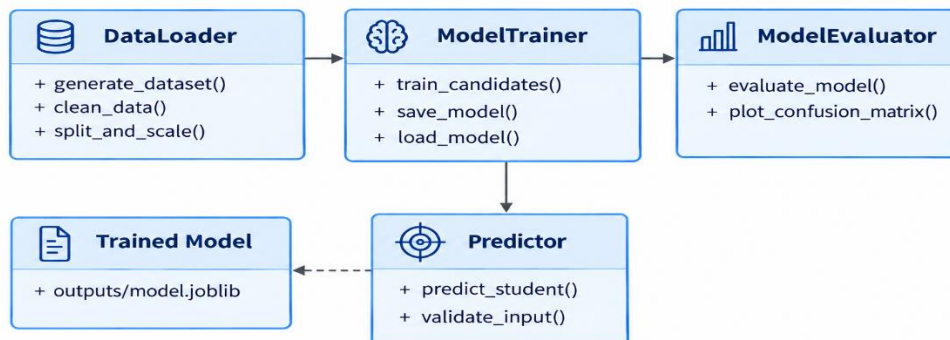
The primary actor (a student or teacher) interacts with the system to load data, run analysis, train/evaluate the model, and get a Pass/Fail prediction.



6.3 Class / Component Diagram

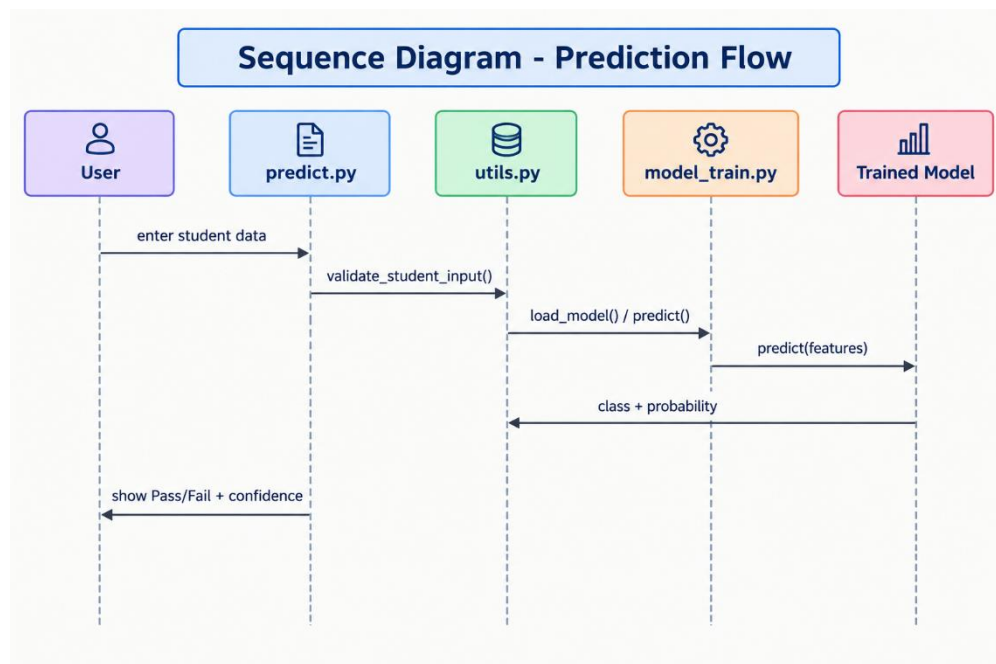
A simplified view of the main components and the key methods each exposes

Class / Component Diagram (simplified)



6.4 Sequence Diagram - Prediction Flow

Shows the sequence of calls when a user requests a prediction for a new student.



7. Design Decisions & Rationale

- **Synthetic dataset:** generated with numpy using a fixed seed, so the project runs fully offline, is reproducible, and avoids licensing issues with external datasets.
- **Two candidate models (Logistic Regression, Decision Tree):** chosen because they represent two different learning paradigms (linear vs. rule-based) taught in the course, and both are fast and interpretable.
- **Automatic model selection** on validation accuracy keeps the pipeline simple while still demonstrating a comparative evaluation step.
- **Feature scaling (StandardScaler):** applied because Logistic Regression is sensitive to feature magnitude, ensuring a fair comparison between models.
- **Modular file structure:** each file has a single responsibility, which keeps the code easy to test, read, and extend (e.g. swapping the synthetic data generator for a real CSV loader requires touching only data_loader.py).

8. Dataset, Model & Evaluation Methodology

8.1 Dataset Description

The dataset contains 500 synthetic student records with four numeric features (study_hours, attendance_pct, previous_score, sleep_hours) and a binary target (result: 0 = Fail, 1 = Pass). The target is derived from a weighted composite of the features plus random noise, so it captures realistic, non-trivial relationships rather than a simple threshold rule.

8.2 Model Selection Rationale

Logistic Regression provides a strong, interpretable linear baseline. Decision Tree captures non-linear feature interactions. Comparing both on the same train/test split demonstrates the standard ML practice of not committing to a single algorithm without empirical comparison.

8.3 Evaluation Methodology

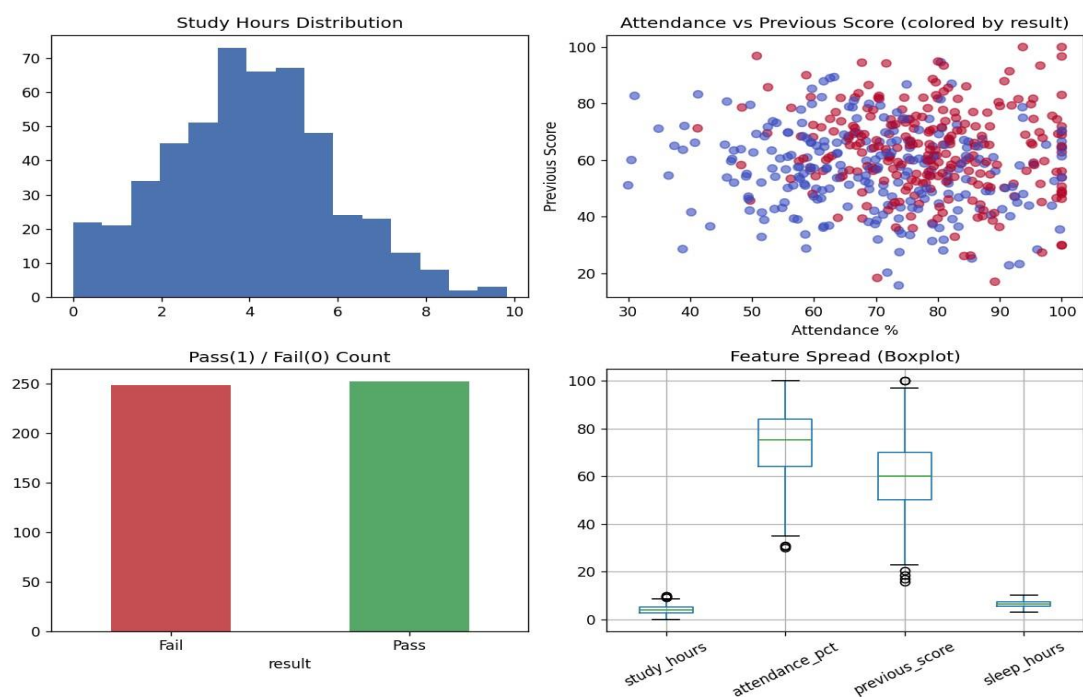
The dataset is split 80/20 into train and test sets (stratified by class) with a fixed random seed. The best model (by test accuracy) is evaluated using accuracy, precision, recall, F1-score, and a confusion matrix, giving a rounded view of performance beyond raw accuracy.

9. Implementation Details

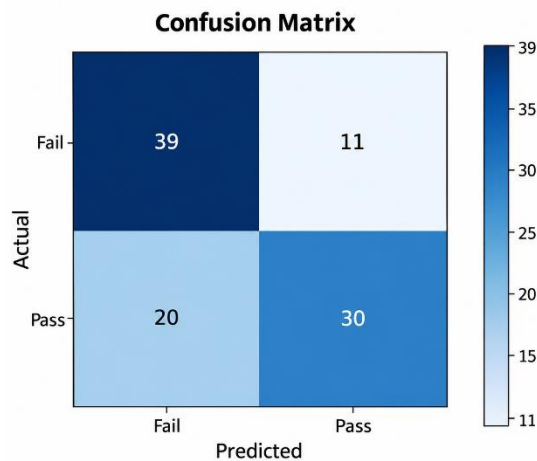
The project is implemented in Python using scikit-learn, pandas, numpy, matplotlib, and joblib. It consists of 8 modules: `utils.py`, `data_loader.py`, `eda.py`, `model_train.py`, `model_evaluate.py`, `predict.py`, `main.py`, and `tests/test_pipeline.py`. `main.py` orchestrates the pipeline by calling each module in sequence. The trained model and scaler are persisted using joblib so predictions can be made instantly without retraining.

10. Screenshots / Results

10.1 Exploratory Data Analysis



10.2 Confusion Matrix



11. Testing Approach

Unit tests (tests/test_pipeline.py, run with pytest) validate: dataset shape and columns, absence of nulls after cleaning, correct train/test split sizes, and both valid and invalid cases for the input-validation function used before prediction. All 6 tests pass, confirming the data pipeline and validation logic behave as expected.

```
PS C:\Users\tanushri\student-performance-predictor> python main.py
===== PIPELINE COMPLETE =====
Best model      : DecisionTree
Accuracy       : 0.690
Precision      : 0.732
Recall         : 0.600
F1-score       : 0.659
Sample predict : {'prediction': 'Pass', 'confidence': 78.16}
Charts & report saved in outputs/

PS C:\Users\tanushri\student-performance-predictor>
```

12. Challenges Faced

- Balancing dataset realism with simplicity: the synthetic data generator needed enough noise to avoid a trivially perfect classifier while still reflecting a believable relationship between habits and outcomes.
- Choosing between multiple candidate models required building a fair comparison pipeline (same split, same scaling) rather than tuning one model in isolation.
- Keeping the code modular while ensuring each module could still be run/tested independently of the full pipeline.

13. Learnings & Key Takeaways

- Practical experience with the full supervised learning workflow: data prep, scaling, training, comparison, and evaluation.
- The importance of feature scaling for models like Logistic Regression.
- How to structure an ML project into clean, testable, single-responsibility modules.
- The value of automated tests even in a data science project, for catching pipeline regressions early.

14. Future Enhancements

- Replace the synthetic dataset with a real, anonymized student dataset.
- Add more models (e.g. Random Forest, SVM) and hyperparameter tuning (GridSearchCV).
- Build a simple web interface (Flask/Streamlit) instead of a CLI for predictions.
- Add cross-validation for a more robust performance estimate.
- Track prediction history and support batch predictions from a CSV upload.

15. References

- scikit-learn documentation - <https://scikit-learn.org/stable/>
- pandas documentation - <https://pandas.pydata.org/docs/>
- matplotlib documentation - <https://matplotlib.org/stable/contents.html>
- Course material: Fundamentals of AI & ML (VITyarthi)